

PROYECTO INTEGRADOR

SecureGen

Generador Inteligente de Contraseñas Seguras

“El impacto de las nuevas tecnologías en la sociedad: desarrollo y proyección de soluciones informáticas”

Nombre:

Camila Cumbicus Betsabe Cuesta

Asignatura: Programación — Estructuras de Datos y Funciones

UIDE | 27 de junio de 2026



El problema

Contraseñas débiles en una sociedad hiperconectada



Contraseñas predecibles

Muchas personas reutilizan claves simples, cortas o fáciles de adivinar en varias plataformas.



Decenas de cuentas digitales

Cada usuario administra correo, redes sociales, banca y plataformas educativas, entre otras.



Riesgo creciente y medible

La dependencia tecnológica multiplica la exposición a accesos no autorizados; las pruebas de SecureGen (ver sección Resultados) muestran cómo una validación simple de longitud y tipo de carácter reduce ese riesgo de forma verificable.



“Una contraseña segura es la primera línea de defensa digital.”

El NIST (2025) recomienda priorizar contraseñas largas y aleatorias por sobre reglas de complejidad forzadas — el criterio adoptado por el validador de SecureGen.

Alcance del proyecto y objetivos

Un problema acotado y verificable: foco deliberado frente a un enfoque genérico

Alcance acotado: consola en Python, sin internet ni dependencias externas (solo random y string), enfocada en 3 funciones verificables: generar, validar y consultar.



Analizar el impacto de las nuevas tecnologías en la sociedad.



Aplicar diseño, programación y arquitectura de software.



Desarrollar un sistema funcional basado en una problemática definida.



Integrar los contenidos de las cuatro unidades de la asignatura.



Reflexionar sobre el uso y futuro de la tecnología en el entorno real.

Cronograma de desarrollo

Planificación del proyecto a lo largo de las 8 semanas del curso

1

Semana 1

Planificación: definición del problema y alcance

2

Semana 2

Investigación sobre impacto tecnológico y ciberseguridad

3

Semana 3

Diseño de diagramas de funcionalidad (casos de uso y flujo)

4

Semana 4

Diseño de la arquitectura del sistema y validaciones

5

Semana 5

Desarrollo del código base (condicionales y repetitivas)

6

Semana 6

Integración de estructuras de datos y funciones (Unidad 4)

7

Semana 7

Pruebas sistemáticas, control de versiones en GitHub

8

Semana 8

Documento final, presentación y video demostrativo

Relación con las 4 unidades de la asignatura

La integración no fue acumulativa, sino funcional: cada estructura aprendida cumple un rol específico

1

Lógica de programación y sintaxis básica de Python (variables, tipos de datos, input()/print()).

2

Estructuras condicionales (if / elif / else) para validar opciones del menú y preferencias.

3

Estructuras repetitivas (while, for) y manejo de errores (try/except) para validar la longitud, la entrada numérica y recorrer el historial.

4

Estructuras de datos (tupla, lista, diccionario) y funciones con parámetros y retorno: el núcleo del sistema.

Unidad 4: Estructuras de Datos y Funciones

Cómo se aplicaron los contenidos en el código de SecureGen (main.py)

Tupla

```
longitudes_recomendadas
```

Valores fijos e inmutables (8,12,16,20) que se muestran como referencia al usuario.

Diccionario

```
caracteres = {...}
```

Agrupar por clave cada conjunto de caracteres disponible; permite activar/desactivar tipos.

Lista

```
historial = []
```

Estructura mutable que crece con `historial.append()` en cada generación.

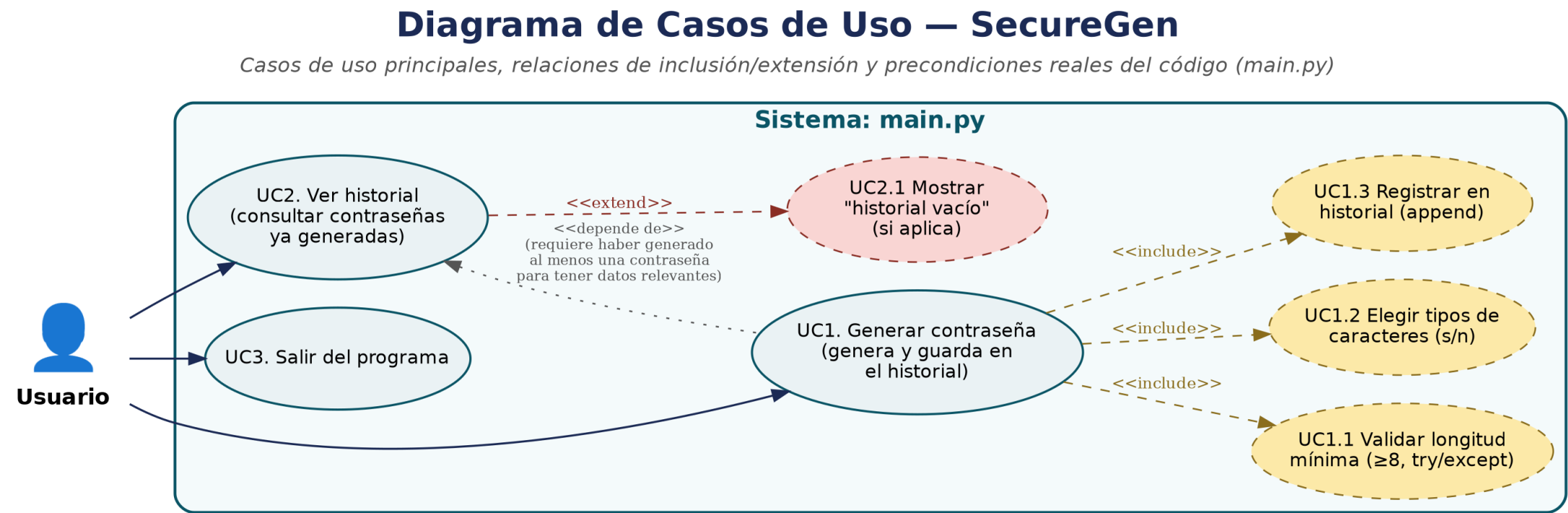
Funciones

```
generar_password(),  
mostrar_historial(),  
menu()
```

Bloques reutilizables con parámetros, validación interna y valor de retorno.

Diagrama de casos de uso

Casos de uso principales y relaciones de inclusión / extensión reales del menú de main.py



Cada caso de uso incluye sub-pasos verificables: generar contraseña requiere validar longitud, elegir tipos de caracteres y registrar el resultado; consultar el historial se extiende con el caso “historial vacío”.

Diagrama de flujo

Lógica completa de menu(), incluyendo validaciones try/except y la construcción de la contraseña

Diagrama de Flujo Detallado — SecureGen (main.py)
Incluye validaciones, manejo de excepciones y la lógica interna de generar_password() y mostrar_historial()

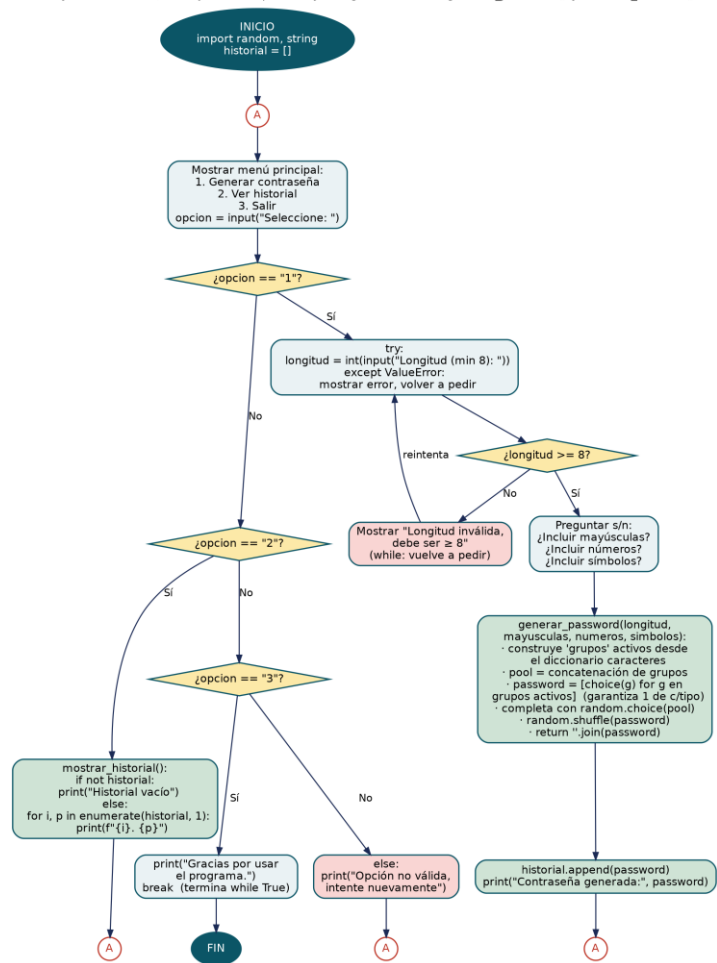
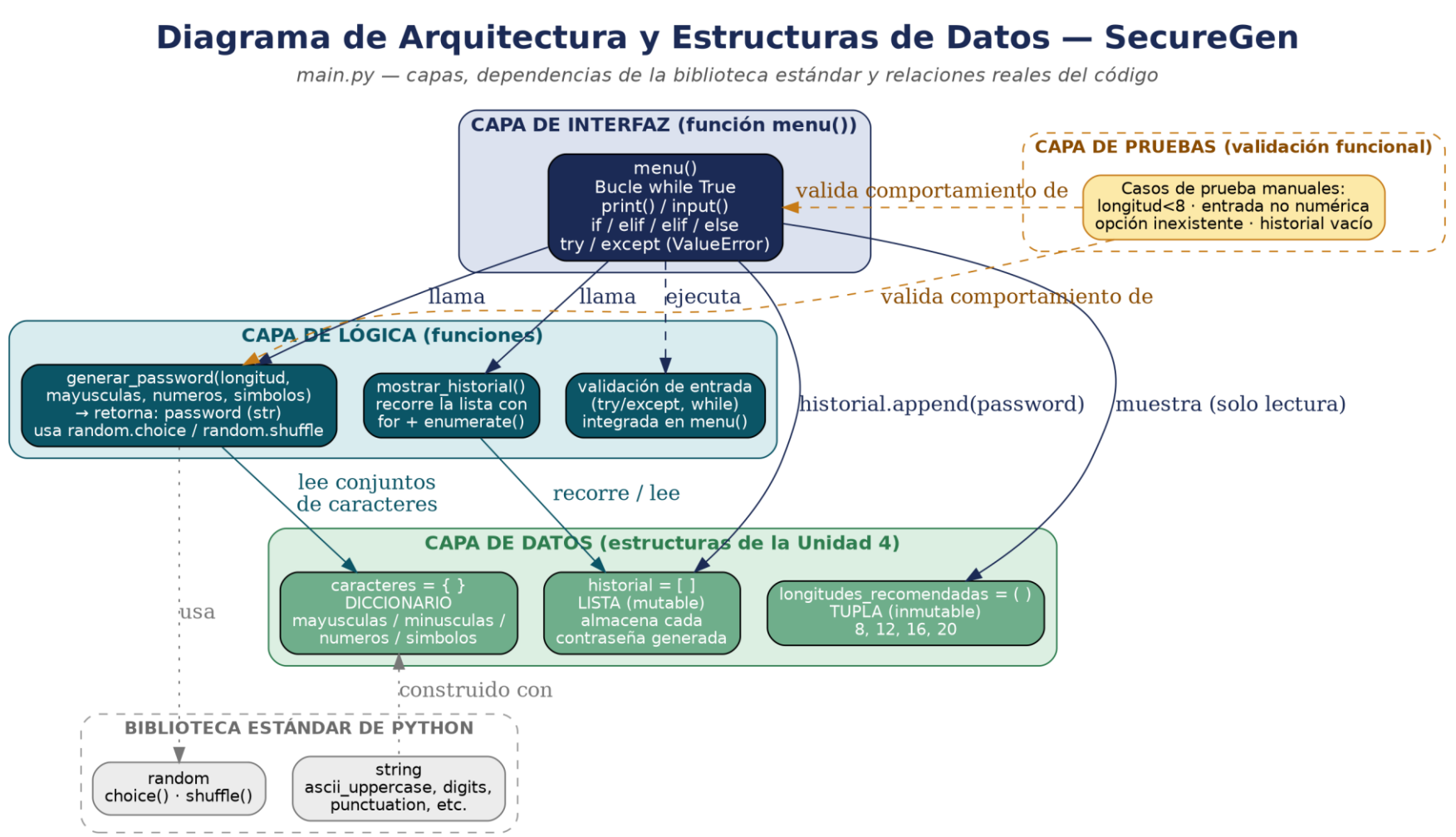


Diagrama de arquitectura

Capas del sistema, dependencias de la biblioteca estándar y estructuras de datos asociadas



Funcionalidades del sistema

main.py — un único archivo, sin librerías externas (solo random y string)



1. Generar contraseña

Solicita longitud (mínimo 8, con reintento si no cumple) y preferencias de mayúsculas, números y símbolos; construye la contraseña con `generar_password()`, garantizando al menos un carácter de cada tipo elegido.



2. Ver historial

Recorre la lista 'historial' con `mostrar_historial()` y muestra cada contraseña generada en la sesión; informa explícitamente si el historial está vacío.



3. Salir

Finaliza la ejecución del programa de manera controlada (rompe el bucle `while True`) tras confirmar la opción 3.

Análisis del software

¿Por qué una tupla, una lista y un diccionario, y no una sola estructura?

🔗 Patrones y diferencias identificadas

- Mutabilidad: la lista 'historial' crece en cada generación; la tupla 'longitudes_recomendadas' nunca cambia.
- Agrupación por clave: el diccionario 'caracteres' permite activar o desactivar conjuntos de forma independiente.
- Iterabilidad compartida: lista y diccionario se recorren con for, pero con propósitos distintos (mostrar vs. construir).
- Alternativa descartada: concatenar todo en una sola cadena de texto impediría elegir qué tipos de caracteres incluir y garantizar variedad mínima.

Conclusión del análisis

La combinación de tupla + diccionario + lista resultó la opción más flexible y legible para este problema, frente a usar una sola estructura genérica. La elección respondió en cada caso a si el dato debía cambiar, agruparse por clave, o permanecer fijo.

Casos de prueba

Verificación sistemática del comportamiento de `generar_password()` y `menu()`

#	Entrada / acción	Resultado esperado	Resultado
1	Longitud = 14, mayúsc./núm./símb. = sí	Contraseña de 14 caracteres con los 4 tipos	Cumplido
2	Longitud = 5 (menor al mínimo)	Mensaje de error y nueva solicitud	Cumplido
3	Longitud = "abc" (texto, no numérico)	ValueError controlado (try/except)	Cumplido
4	Opción de menú = "9" (no existe)	Mensaje "Opción no válida"	Cumplido
5	Generar 3 contraseñas y elegir "Ver historial"	Se listan las 3, en orden, con enumerate()	Cumplido
6	"Ver historial" antes de generar ninguna	Mensaje "Historial vacío", sin error	Cumplido

Resultados

Conclusiones basadas directamente en la evidencia de las pruebas ejecutadas

Conclusión de las pruebas

- ✓ Genera contraseñas que siempre respetan la longitud mínima solicitada.
- ✓ Incluye correctamente los tipos de caracteres seleccionados por el usuario.
- ✓ Registra de forma fiel cada contraseña generada en la lista 'historial'.
- ✓ Responde sin interrumpirse ante entradas no válidas (texto, opciones inexistentes).

Extrapolación lógica

La estrategia de separar configuración (diccionario), historial mutable (lista) y referencias fijas (tupla), junto con validación try/except aislada, es aplicable a otros sistemas de consola de escala comparable (gestor de tareas, encuestas, validadores de formularios). No se extiende a sistemas de ciberseguridad de mayor escala, fuera del alcance verificado.

Implicaciones y limitaciones

Discusión detallada y evidenciada a partir del comportamiento real del código

✓ Implicaciones

- Reduce prácticas de riesgo evidenciadas en pruebas (longitudes insuficientes, omisión de tipos de caracteres), al forzar una decisión explícita del usuario.
- La separación verificada interfaz/lógica/datos demuestra en código que conceptos básicos pueden traducirse en una solución mantenible.
- El manejo de excepciones probado (Tabla de casos 2 y 3) evidencia robustez sin frameworks externos.

✗ Limitaciones

- El historial solo existe en memoria durante la sesión; no hay persistencia entre ejecuciones.
- El módulo random no es criptográficamente seguro (se recomienda 'secrets' para producción).
- No verifica la contraseña contra bases de credenciales filtradas, práctica recomendada por el NIST (2025).
- La interfaz de consola limita su uso a personas sin perfil técnico; la fortaleza final depende de las opciones que elija el usuario.



Reflexión

“La programación, incluso a nivel introductorio, puede convertirse en una herramienta concreta para enfrentar problemas sociales actuales como la ciberseguridad.”

Proteger la identidad digital ya no es un tema exclusivo de especialistas en informática: es una necesidad de toda la ciudadanía en la sociedad actual.

Conclusiones

- ▶ Se desarrolló un sistema funcional que integra tuplas, listas, diccionarios y funciones, dentro de un alcance deliberadamente acotado y verificable.
- ▶ Los diagramas de casos de uso, flujo y arquitectura — con sus relaciones de inclusión, extensión y dependencias — facilitaron la planificación antes de programar.
- ▶ Se reforzaron buenas prácticas: código comentado, funciones reutilizables, validación de entrada y manejo de errores.
- ▶ Las pruebas sistemáticas confirman el cumplimiento de los objetivos y permiten una extrapolación lógica acotada a sistemas de escala comparable.
- ▶ Se reconocieron limitaciones concretas y evidenciadas, lo que demuestra una evaluación crítica y realista del trabajo.

Referencias

Luján Castillo, M. (2019). Estructuras de datos en Python. Material de la asignatura, Unidad 4: Estructura de Datos y Funciones, UIDE.

National Institute of Standards and Technology. (2025). Digital identity guidelines: Authentication and authenticator management (NIST SP 800-63B, Rev. 4). U.S. Department of Commerce.

Python Software Foundation. (2024). The Python standard library. <https://docs.python.org/3/library/>

UIDE. (2026). Guía del proyecto integrador: Programación [Documento institucional de la asignatura].



Gracias

SecureGen — Generador Inteligente de Contraseñas Seguras

[Repositorio del proyecto disponible en GitHub](#) | [Video demostrativo adjunto](#)